Transport Layer (2, 3, 4)

COMP90007 Internet Technologies

Lecturer: Adel N. Toosi

Outline

- Transport Layer Service
- Elements of Transport Protocols
- Programming using Sockets
- Transport Layer Protocols: TCP and UDP
- Quality of Service

Programming using Sockets

- Sockets widely used for interconnections
  - Socket: transport endpoints
  - “Berkeley" sockets are predominant in Internet applications
  - Simple set of primitives plus SOCKET, BIND, and ACCEPT

Primitive	Meaning
SOCKET	Create a new communication end point
BIND	Associate a local address with a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Passively establish an incoming connection
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

Recall Pseudo Code Example

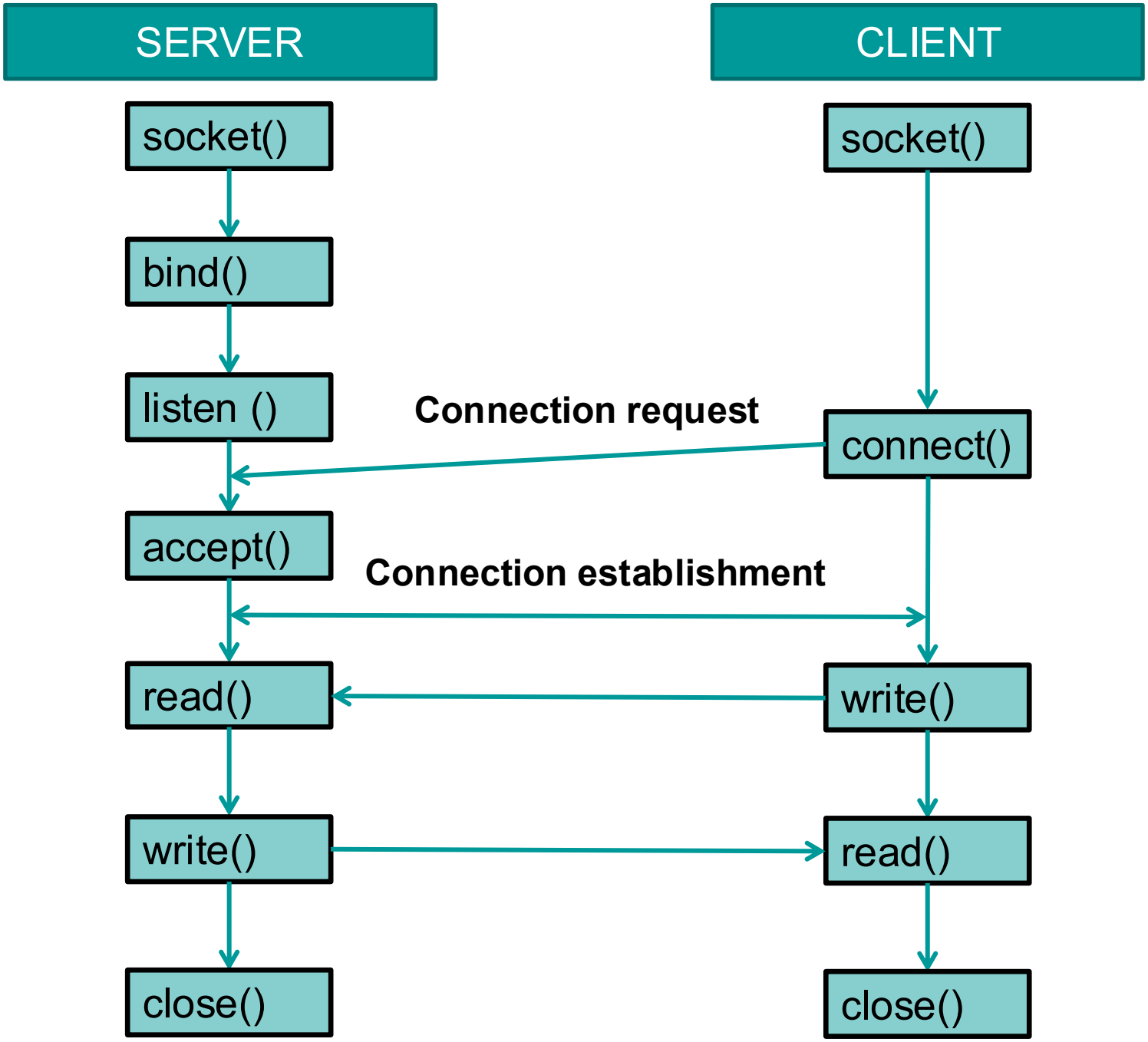
```
Socket A_Socket = createSocket("TCP");

connect(A_Socket, 128.255.16.0, 80);

send(A_Socket, “My first message!”);

disconnect(A_Socket);
```

… there is also a server component for this client that runs on another host…


SERVER
CLIENT
socket()
socket()
bind()
listen ()
Connection request
connect()
accept()
Connection establishment
read()
write()
write()
read()
close()
close()

Socket Example – Client Side

```
s = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
if (s <0) fatal("socket");
memset(&channel, 0, sizeof(channel));
channel.sin_family= AF_INET;
memcpy(&channel.sin_addr.s_addr, h->h_addr, h->h_length);
channel.sin_port= htons(SERVER_PORT);

c = connect(s, (struct sockaddr *) &channel, sizeof(channel));
```

Note: the example from the book has more details but the essence is the same

Socket Example – Server Side

```
memset(&channel, 0, sizeof(channel));
channel.sin_family = AF_INET;
channel.sin_addr.s_addr = htonl(INADDR_ANY);
channel.sin_port = htons(SERVER_PORT);

s = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
if (s < 0) fatal("socket failed");
setsockopt(s, SOL_SOCKET, SO_REUSEADDR, (char *) &on, sizeof(on));
b = bind(s, (struct sockaddr *) &channel, sizeof(channel));
if (b < 0) fatal("bind failed");
l = listen(s, QUEUE_SIZE);
if (l < 0) fatal("listen failed");

. . .
```

Assign address

Prepare for incoming connections

Socket Example – Server Side

Block waiting for the next connection

```
while (1) {
    sa = accept(s, 0, 0);
    if (sa < 0) fatal("accept failed");
    read(sa, buf, BUF_SIZE);
    /* Get and return the file. */
    fd = open(buf, O_RDONLY);
    if (fd < 0) fatal("open failed");
    .....
    close(fd);          /* close file */
    close(sa);          /* close connection */
}
```

Read request

python Copy code

```python
import socket

def start_echo_server(host='127.0.0.1', port=65432):
    # Create a TCP/IP socket
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
        # Bind the socket to the address and port
        server_socket.bind((host, port))
        # Listen for incoming connections
        server_socket.listen()
        print(f"Server listening on {host}:{port}")

        # Accept a connection
        conn, addr = server_socket.accept()
        with conn:
            print(f"Connected by {addr}")
            while True:
                # Receive data from the client
                data = conn.recv(1024)
                if not data:
                    break
                print(f"Received from client: {data.decode()}")
                # Send the data back to the client
                conn.sendall(data)

if __name__ == "__main__":
    start_echo_server()
```

python Copy code

```python
import socket

def start_echo_client(host='127.0.0.1', port=65432):
    # Create a TCP/IP socket
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
        # Connect to the server
        client_socket.connect((host, port))
        print(f"Connected to server {host}:{port}")

        while True:
            # Get user input
            message = input("Enter message to send (type 'exit' to quit): ")
            if message.lower() == 'exit':
                break

            # Send the message to the server
            client_socket.sendall(message.encode())

            # Receive the response from the server
            data = client_socket.recv(1024)
            print(f"Received from server: {data.decode()}")

if __name__ == "__main__":
    start_echo_client()
```

Socket Example – Multi-Threading

- The server can create a new thread to handle the connection on the new socket and go back to waiting for the next connection on the original socket…

```
ServerSocket serverSocket = new ServerSocket([parameters]);


While (true) {
        Socket socket = serverSocket.accept();
        MultiThreadMyServer server = new MultiThreadMyServer();
        server.setMyService([some more parameters]);
        server.setSocket(socket);
        new Thread(server).start();
    ....
```

Ref: Code from OO Programming with Java; Chp. 14

Socket Example – Multi-Threading

More info on **threads**…

```
class MultiThreadMyServer extends Thread {
        int somedata;
        MultiThreadMyServer() {
            this.somedata = …;
            …
        }

            … more methods here

        public void run() {
             . . .
        }
    }
```

Outline

- Transport Layer Service
- Elements of Transport Protocols
- Programming using Sockets
- Transport Layer Protocols: TCP and UDP
- Quality of Service

User Datagram Protocol

- Connectionless service
  - Does not add much to the Network Layer functionality
  - Reliability issue?

    Left to the application layer, including retransmission decisions as well as congestion control
  - UDP is used for apps like video streaming/gaming regularly

Example: UDP Client Side

```
public static void main(String args[]) {
        ....
        DatagramSocket mySocket = new
                DatagramSocket();
        mySocket.send([data,address, etc
                parameters]);
        …
}
```

Example: UDP Server Side

```
public static void main(String args[]) {
        ….
        DatagramSocket server = new
                DatagramSocket(port);
        while (true) {
                server.receive([parameters]);
                …
        }}
```

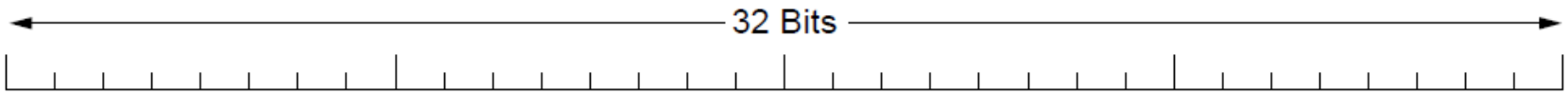
UDP Segment

- Provides a protocol whereby applications can transmit encapsulated IP datagrams without a connection establishment

- UDP segment: an 8-byte header followed by the payload
  - UDP headers contain source and destination ports
  - Payload is handed to the process which is attached to the particular port at destination

UDP Segment

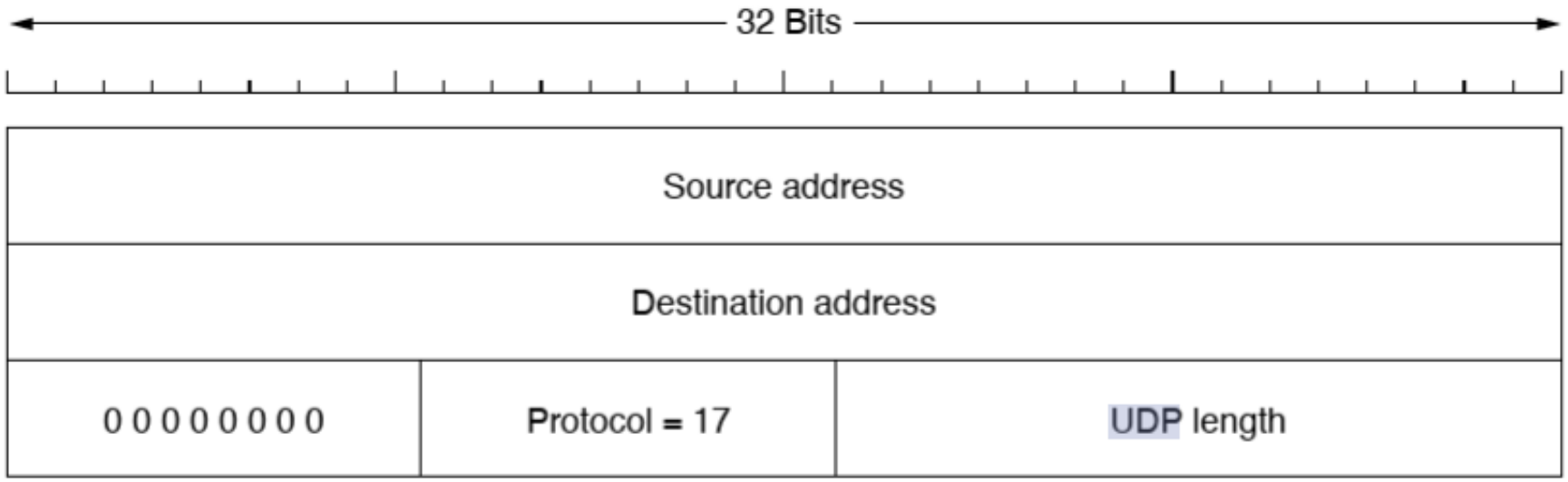
- Structure of UDP header: ports (TSAPs), length and checksum
  - Both source and destination ports are required - destination allows for incoming segments, source allows reply for outgoing segments



Source port	Destination port
UDP length	UDP checksum

UDP Checksum

- Checksum=header, data and a conceptual IP **pseudoheader**
- The pseudoheader for IPv4



Port Numbers

- Transport Service Access Points (TSAPs) = port numbers on the Internet (TCP and UDP)
- Port numbers can range from 0-65535
- Ports are classified into 3 segments:
  - Well-Known Ports (0-1023); Registered Ports (1024-49151); Dynamic Ports (49152-65535)

Port	Protocol	Use
20, 21	FTP	File transfer
22	SSH	Remote login, replacement for Telnet
25	SMTP	Email
80	HTTP	World Wide Web
110	POP-3	Remote email access
143	IMAP	Remote email access
443	HTTPS	Secure Web (HTTP over SSL/TLS)
543	RTSP	Media player control
631	IPP	Printer sharing

http://www.iana.org/assignments/port-numbers

Strengths and Weaknesses of UDP

- **Strengths**: main advantage of using UDP over raw IP?
  - provides an interface with multiplexing/de-multiplexing capabilities, by specifying ports for source and destination pairs, i.e., addressing for processes
- **Weaknesses**: UDP does not include support for *flow control*, *congestion control*, or *retransmission*

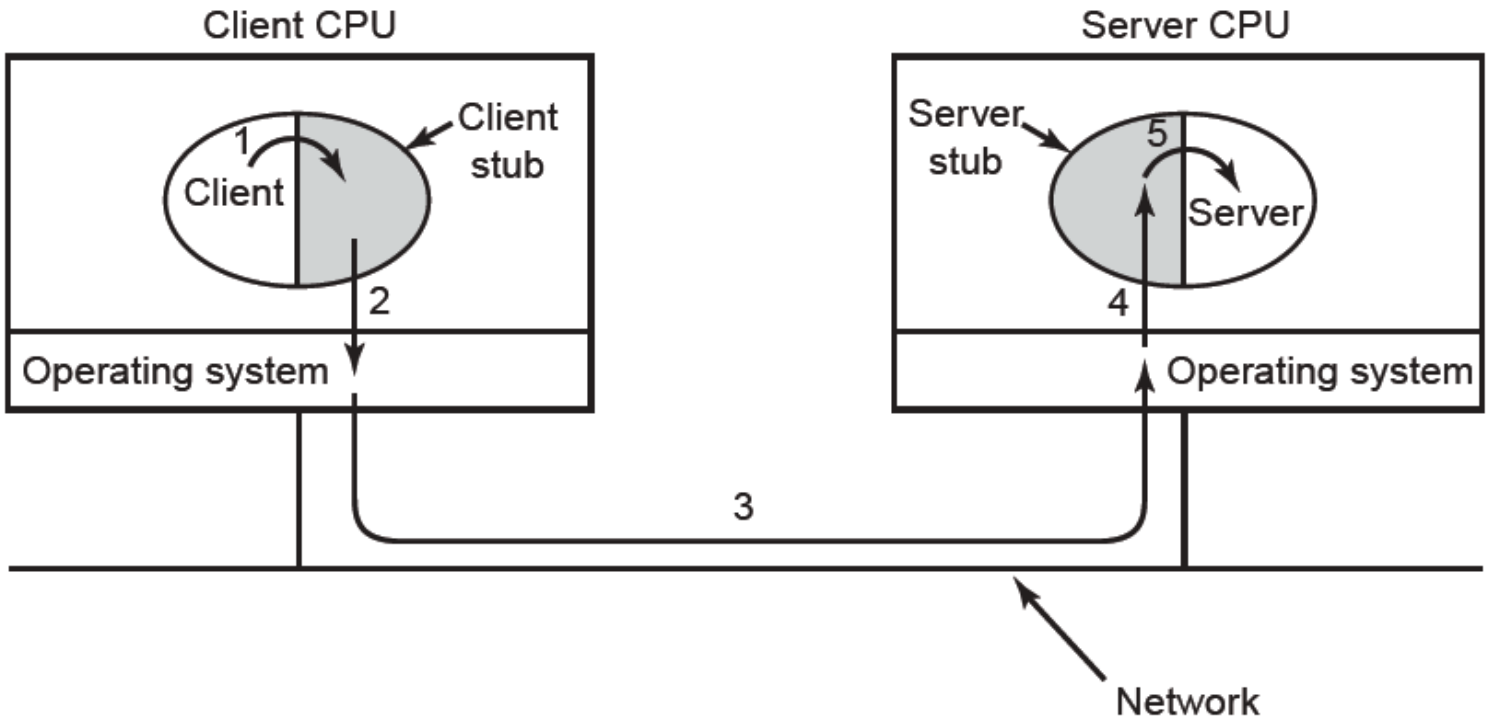
- **Conclusion**: UDP is a good choice if applications require high speed or a precise level of control over packet flow/error/timing

  e.g. Domain Name System over the Internet uses UDP

Example of using UDP

- Remote Procedure Call (RPC)
  - Sending a message and getting a reply back is analogous to making a function call in programming languages
  - Birrell and Nelson proposed RPC to allow programs to call procedures on remote hosts using UDP

Remote Procedure Call



Remote Procedure Call

- To call a remote procedure, the client is bound to a small library (the **client stub**) that represents the server procedure in the client's address space.
- Similarly the server is bound with a procedure called the **server stub**.
- These stubs hide the fact that the procedure itself is not local.

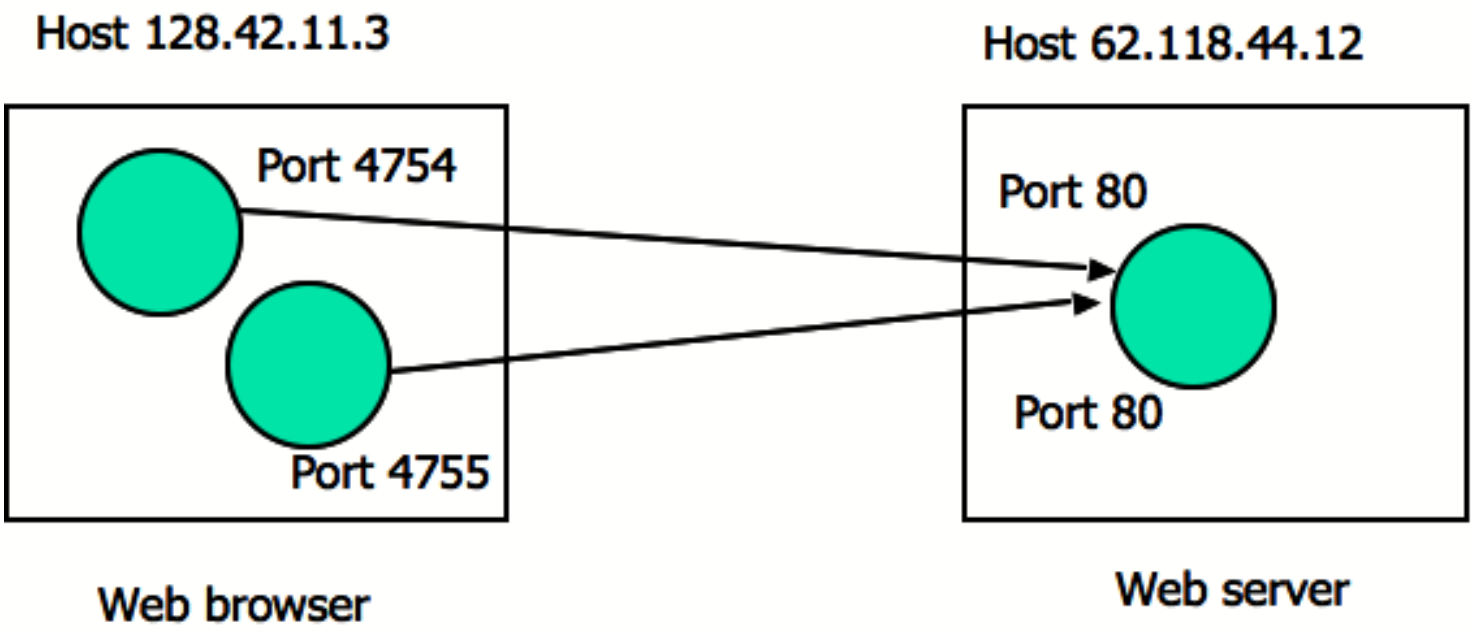
Transmission Control Protocol

- **Connection-oriented** service, increasing **reliability**
- TCP transport entity manages **TCP streams** and interfaces to the IP layer - can exist in numerous locations (kernel, library, user process)
- Sending TCP entity
  - accepts user data streams
  - splits them into pieces < 64KB (often at a size in order so that the IP and TCP headers can fit into a single Ethernet frame)
  - sends each piece as a separate IP datagram
- Recipient TCP entity
  - reconstructs the original byte streams from the encapsulation

TCP Service Model

- Sender and receiver both **create sockets**, consisting of the IP address of the host and a port number
- For TCP Service to be activated, **connections** must be explicitly established between a socket at a sending host (src-host, src-port) and a socket at a receiving host (dest-host, dest-port)
- A server can manage multiple connections simultaneously using a single socket

Example

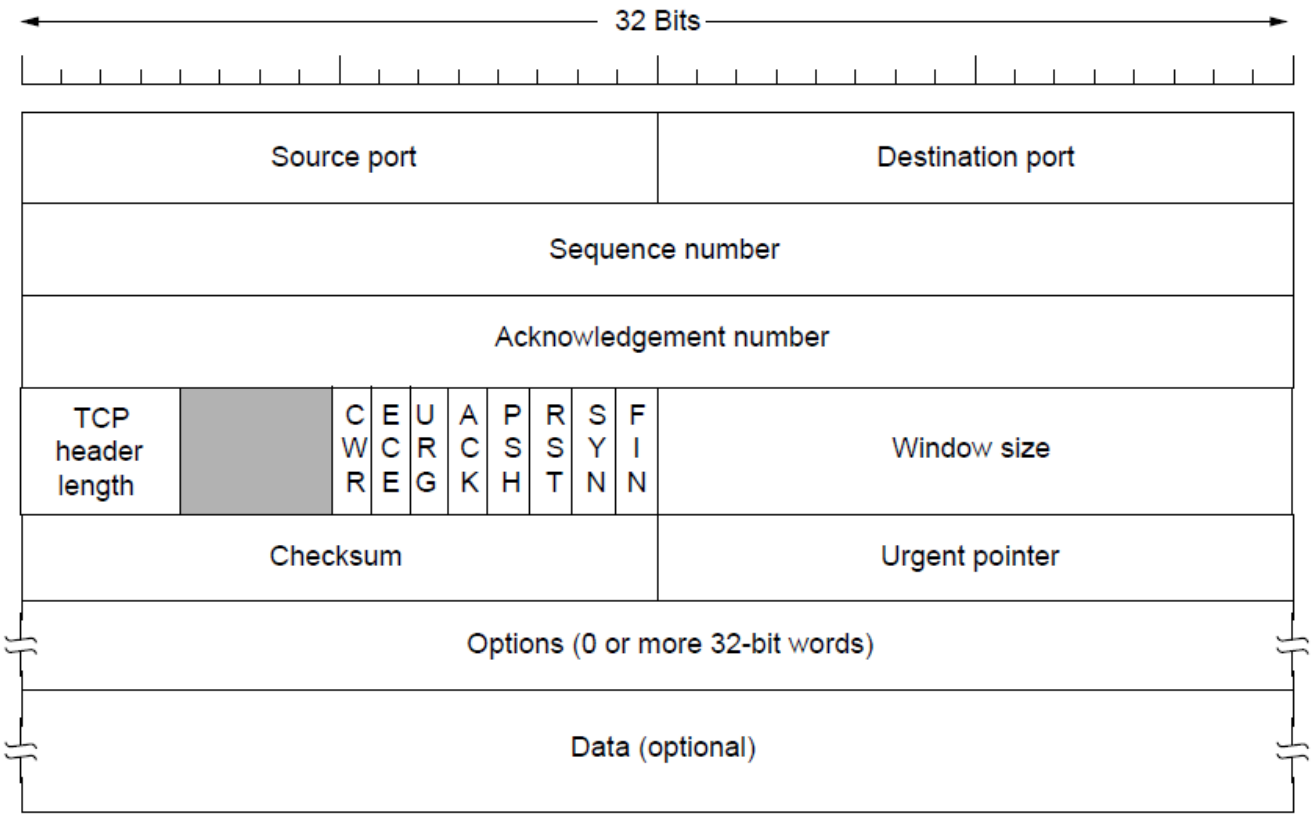

Host 128.42.11.3
Host 62.118.44.12
Port 4754
Port 80
Port 80
Port 4755
Web browser
Web server

TCP Protocol

- Data sent between TCP entities in segments
  - Segment has a 20-byte header plus zero or more data bytes
  - Two limits restrict the segment size:
    - IP packet payload - 65,515 bytes
    - Ethernet frame payload - generally 1,500 bytes
- Sliding window strategy
  - Sender transmits and starts a timer
  - Receiver sends back an acknowledgement which is the **next sequence** number expected and the remaining window size
  - If sender's timer expires before acknowledgement, then the sender transmits the original segment again

TCP Segment Header (1)

- Fixed-format 20-byte header + options + data



TCP Segment Header (2)

- ***Source port* and *Destination port:*** identify the local end points of the connection.
- ***Sequence number and Acknowledgement number:*** perform their usual functions.
- ***TCP header* length:** how many 32-bit words are contained in the TCP header.
- ***Window size:*** how many bytes may be sent starting at the byte acknowledged.
- ***Checksum:*** for extra reliability. It checksums the header and data.
- ***Options:*** add extra facilities not covered by the regular header, e.g. Maximum Segment Size, Window Scale, Timestamp, Selective Acknowledgement.

TCP Segment Header (3)

- ***ACK:*** set to 1 to indicate that the Acknowledgement number is valid, 0 means ignore ACK number field.
- ***SYN:*** establish connections.
  - The **connection request** has SYN = 1 and ACK = 0. The **connection reply** does bear an acknowledgement, so it has SYN = 1 and ACK = 1.
- ***FIN:*** release a connection. It specifies that the sender has no more data to transmit. After closing a connection, the closing process may continue to receive data.
- ***RST:*** abruptly reset a connection that has become confused. It is also used to reject an invalid segment or refuse an attempt to open a connection.

TCP Segment Header (4)

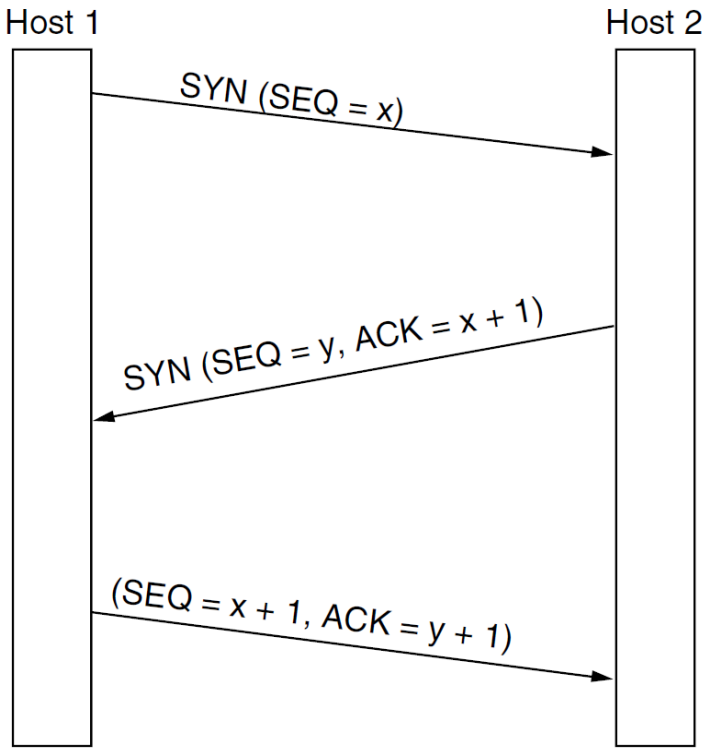
- ***CWR, ECE:*** signal congestion when *ECN* (Explicit Congestion Notification) is used.
  - ***ECE:*** signal an ECN-Echo to a TCP sender to tell it to slow down when the TCP receiver gets a congestion indication from the network.
  - ***CWR:*** signal Congestion Window Reduced from the TCP sender to the TCP receiver so that it knows the sender has slowed down and can stop sending the ECN-Echo.
- ***PSH:*** indicates PUSHed data. The receiver is requested to deliver the data to the application upon arrival and not buffer it.
- ***URG:*** set to 1 if the *Urgent pointer* is in use. The Urgent pointer indicates a byte offset from the current sequence number at which urgent data are to be found.

TCP Connections

- Features of TCP connections:
  - **Full duplex:** data in both directions simultaneously
  - **Point-to-point:** exact pairs of senders and receivers
  - **Byte streams:** not message streams - message boundaries are not preserved
  - **Buffer options:** TCP entity can choose to buffer prior to sending or not depending on the context
    - TCP_NODELAY in Windows and Linux
    - Socket.setTcpNoDelay(boolean)

TCP Connection Management

- **Establishment:** three-way handshake
- **Release**: symmetric release
  - Two simultaneous connection attempts result in only one connection
  - Timers used for lost connection releases



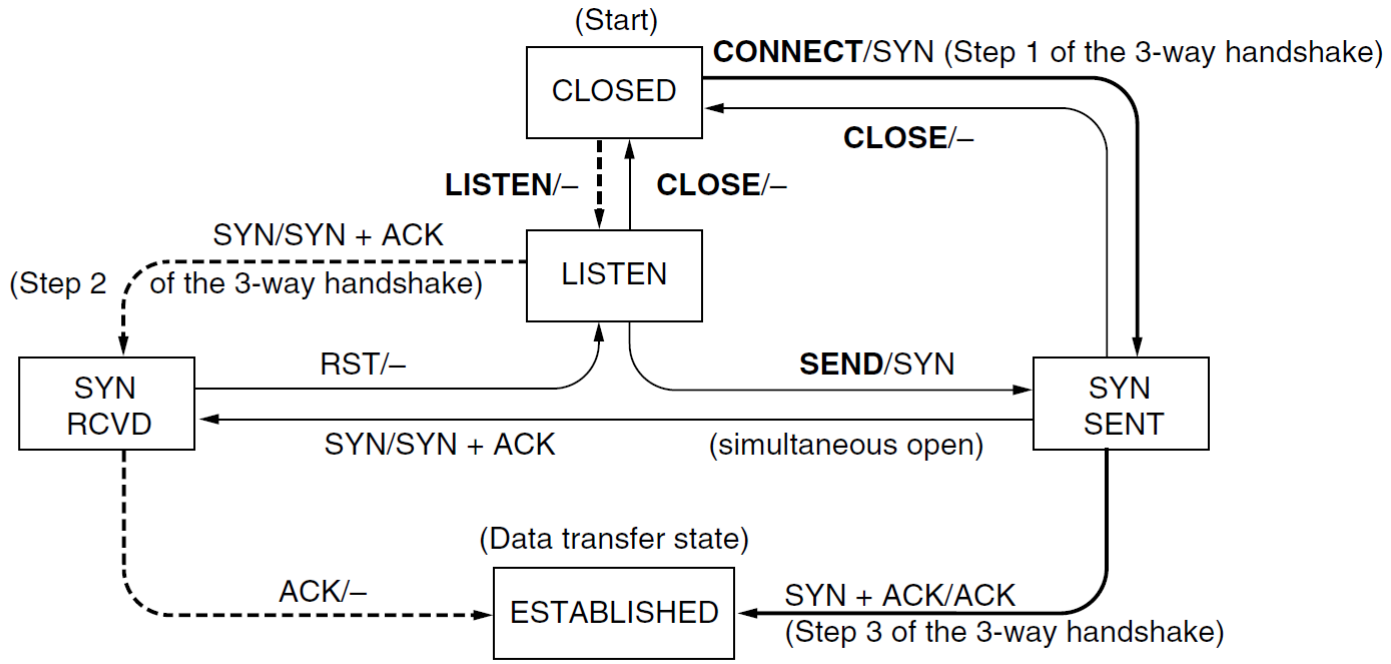
TCP Connection Management

- Finite state machine and state diagram

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIME WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

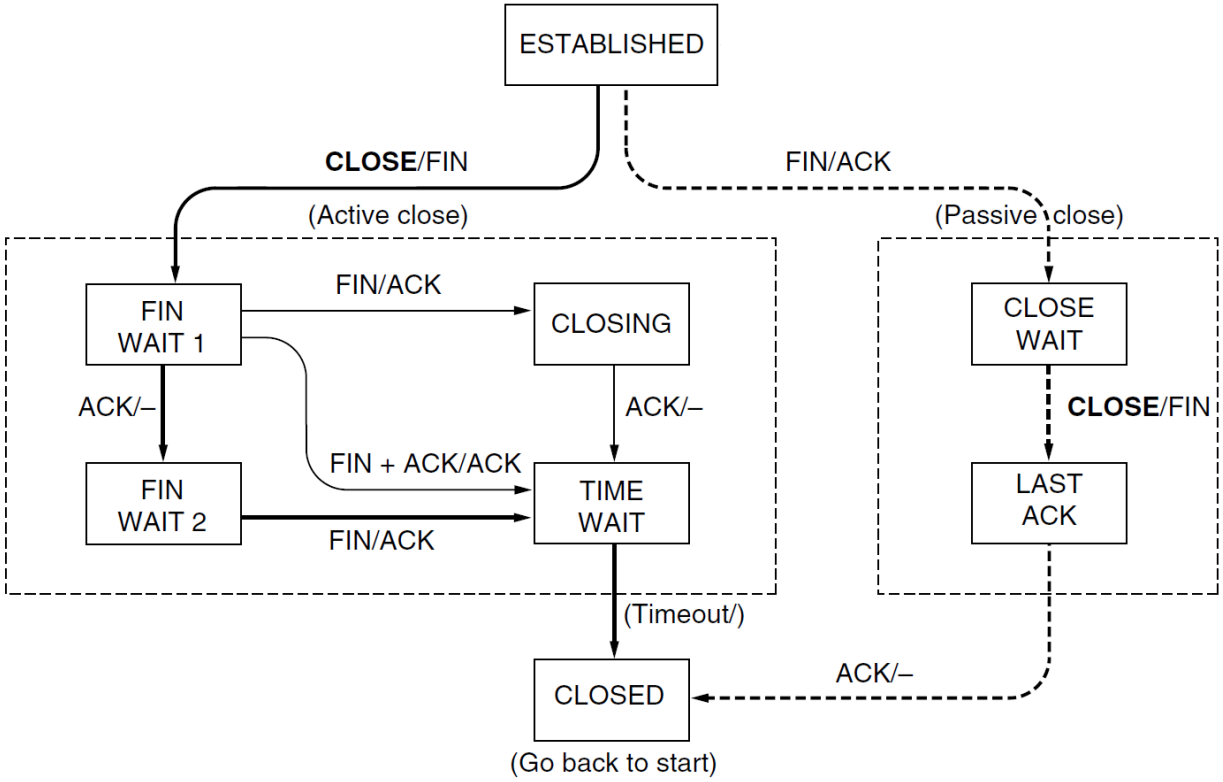
TCP Connection Management

- Connection establishment



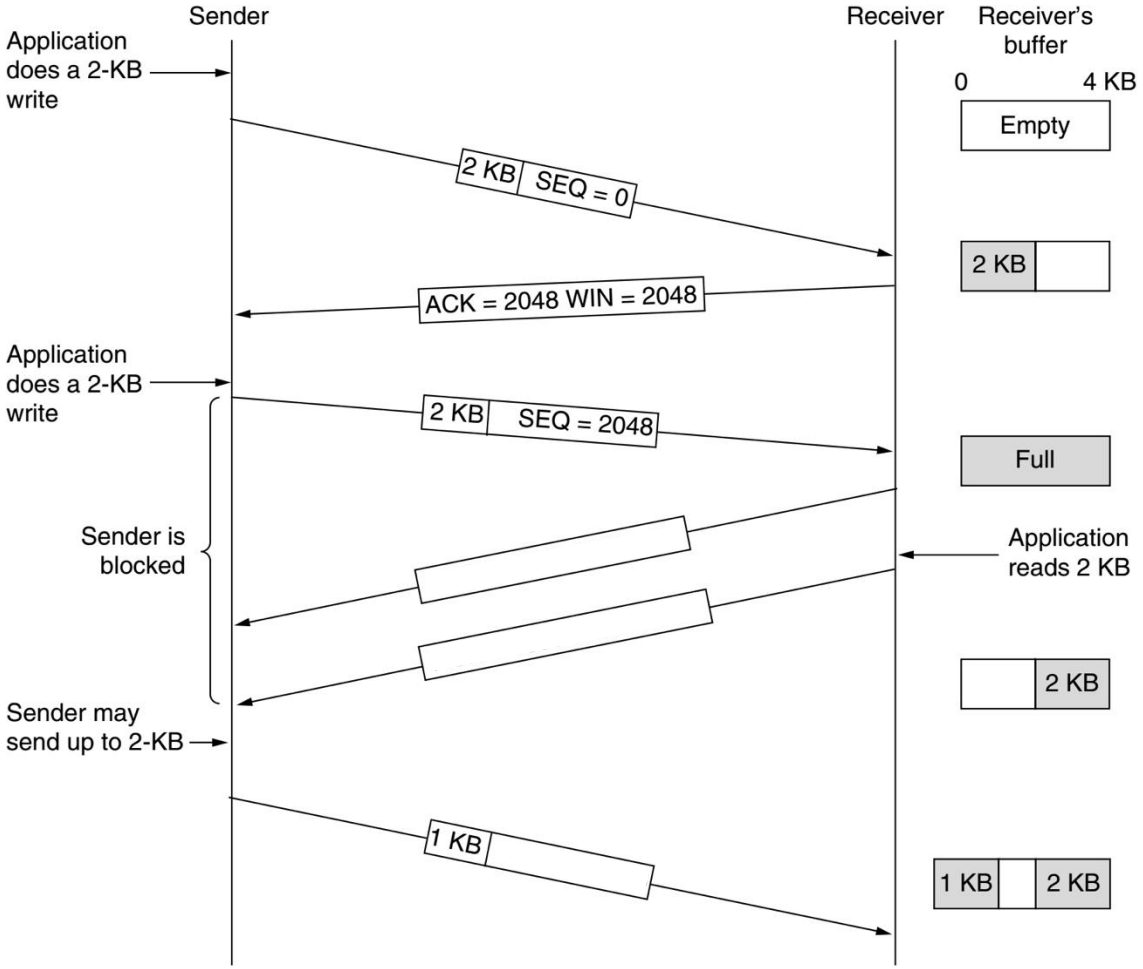
TCP Connection Management

- Connection release



TCP Window Management

- TCP acknowledges the correct receipt of segments
- Receiver advertises window based on available buffer space



TCP Timer Management

- When retransmission timers should go out?
  - Difficult to estimate the round-trip time to the destination
  - Too early means too many resends
  - Too late means reliability comes with more additional cost
- Dynamic solutions that measure network performance and adapt timers
  - Measure smoothed round-trip time (SRTT) and the variation of SRTT

Outline

- Transport Layer Service
- Elements of Transport Protocols
- Transport Layer Protocols: TCP and UDP
- Quality of Service

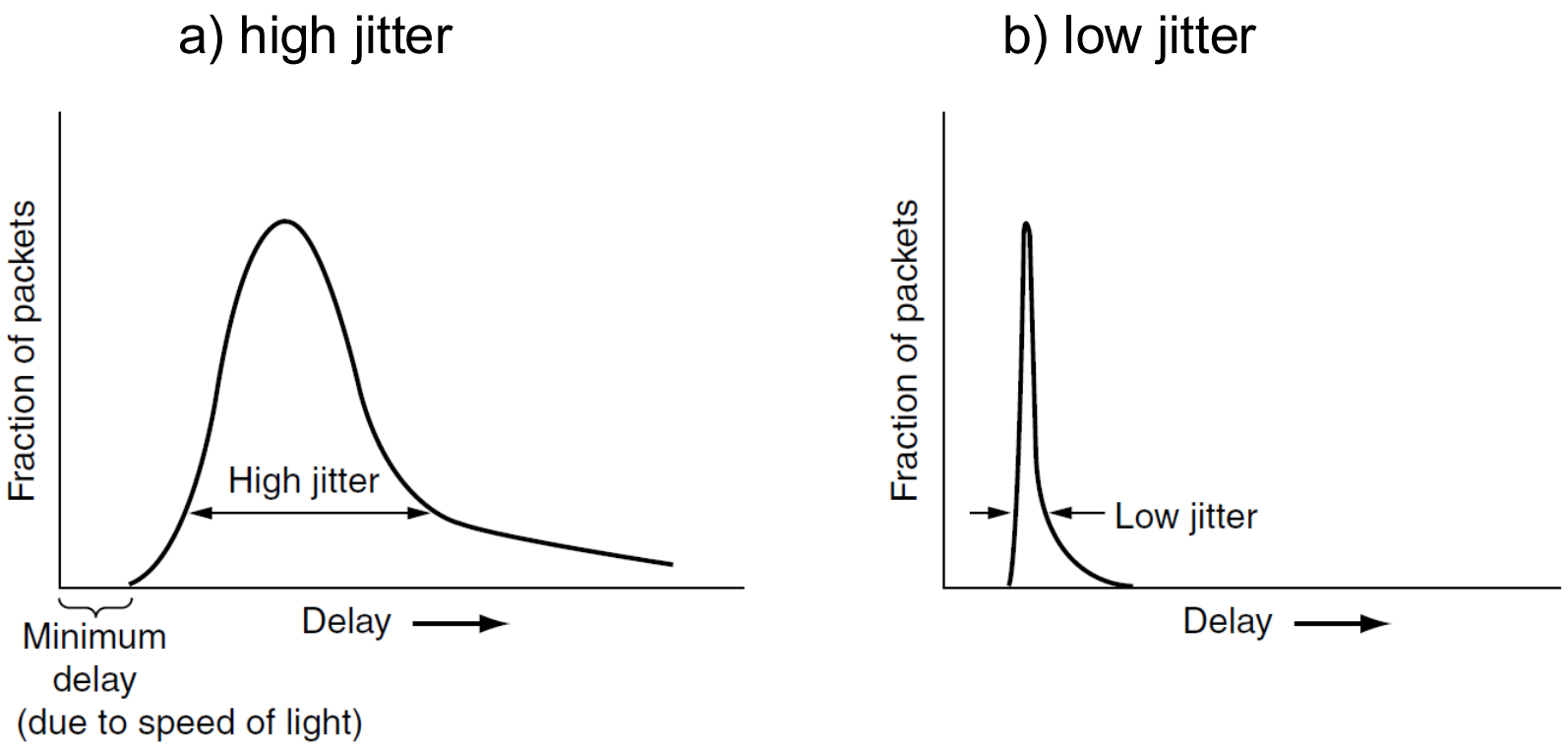
Quality of Service

- What is the key problem if network is not delivering properly?
- Expected network performance is an important criterion for a wide range of network applications
- 4 things to watch out for:

  bandwidth, reliability, delay, jitter

Jitter Control

- Jitter is the **variation** in packet arrival times



Jitter Control

- For certain applications jitter control is extremely important as it may directly affect the quality perceived by the application users

- Approaches to jitter control
  - Buffer packets at the receiver
  - Shuffle transmission: slower packets sent first, faster packets wait in a queue

QoS Requirements

- Different applications care about different properties

Application	Bandwidth	Delay	Jitter	Loss
Email	Low	Low	Low	Medium
File sharing	High	Low	Low	Medium
Web access	Medium	Medium	Low	Medium
Remote login	Low	Medium	Medium	Medium
Audio on demand	Low	Low	High	Low
Video on demand	High	Low	High	Low
Telephony	Low	High	High	Low
Videoconferencing	High	High	High	Low

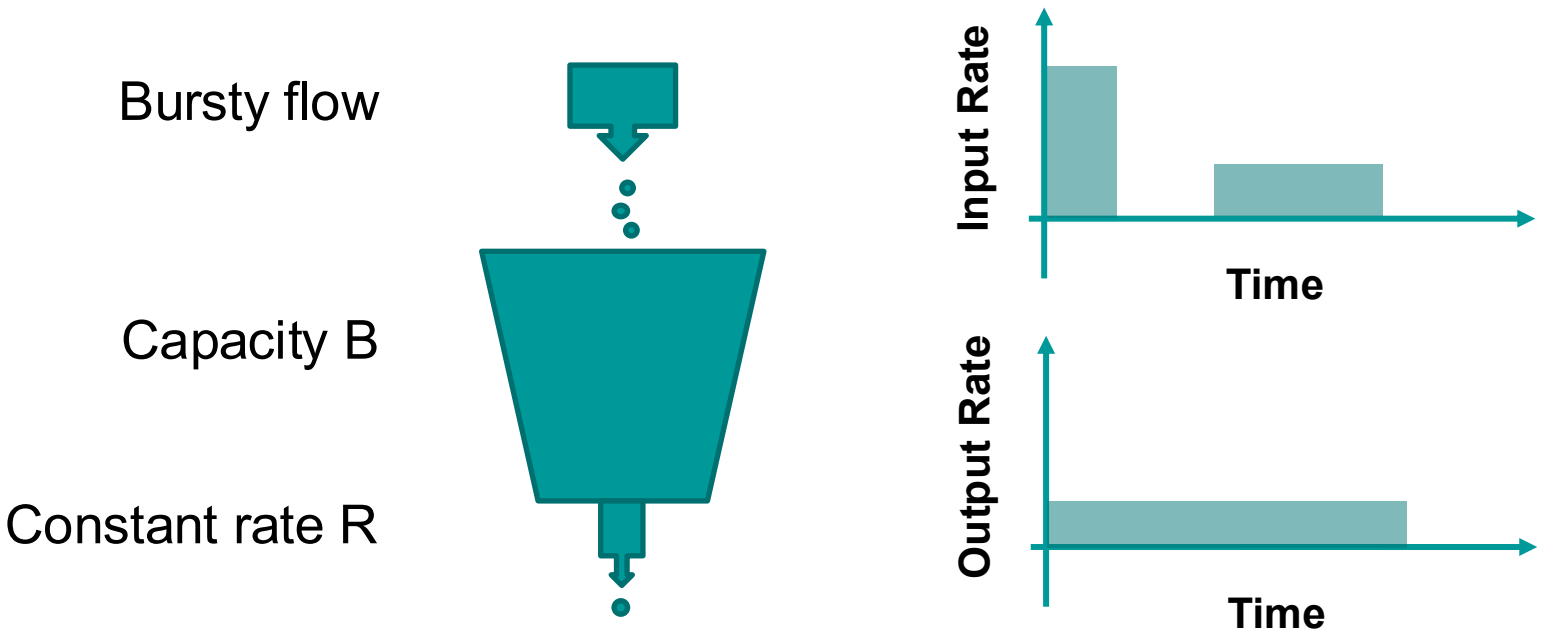
Note: “High” means a demanding requirement!

Techniques for Achieving QoS

- **Over-provisioning**
  - more than adequate buffer, router CPU, and bandwidth
- **Buffering**
  - buffer received flows before delivery - increases delay, but smoothes out jitter, no effect on bandwidth
- **Traffic Shaping**
  - regulate the average rate of transmission and burstiness of transmission, e.g. leaky bucket

Leaky Bucket

- Large bursts of traffic is buffered and smoothed while sending, which can be done at the sender

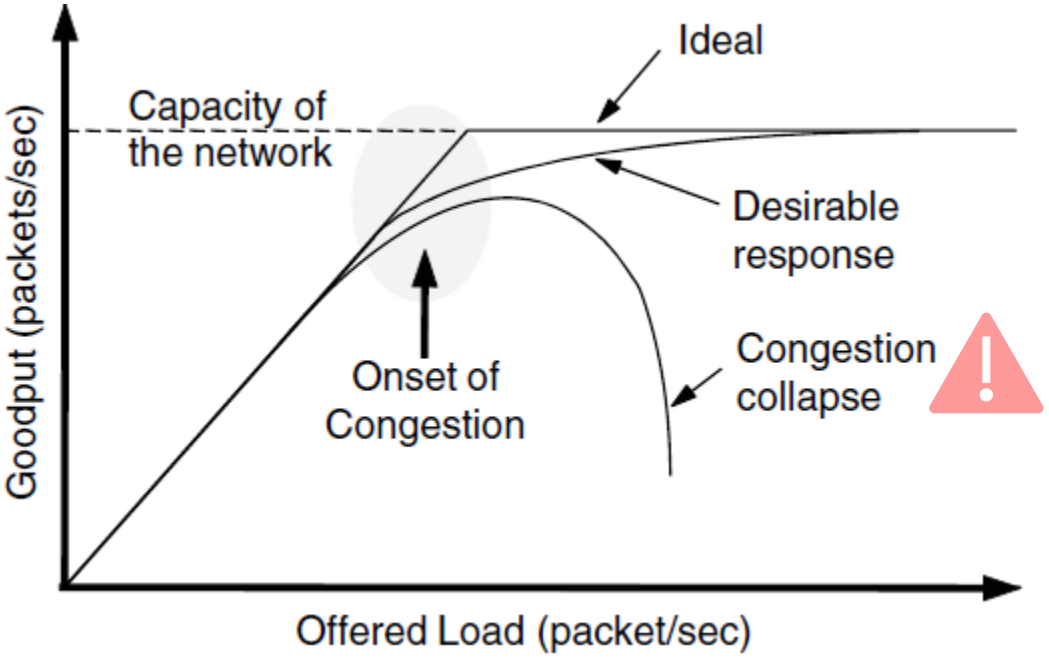


Techniques for Achieving QoS

- **Resource reservation**
  - reserve bandwidth, buffer space, CPU in advance
- **Admission control**
  - routers can decide based on traffic patterns whether to accept new flows, or reject/reroute them
- **Proportional routing**
  - traffic for same destination split across multiple paths
- **Packet scheduling**
  - create queues based on priority, e.g. fair queueing, weighted fair queueing

What happens when congested?

- Congestion results when too much traffic is offered; performance degrades due to loss/retransmissions
- Goodput (=useful packets) vs. offered load

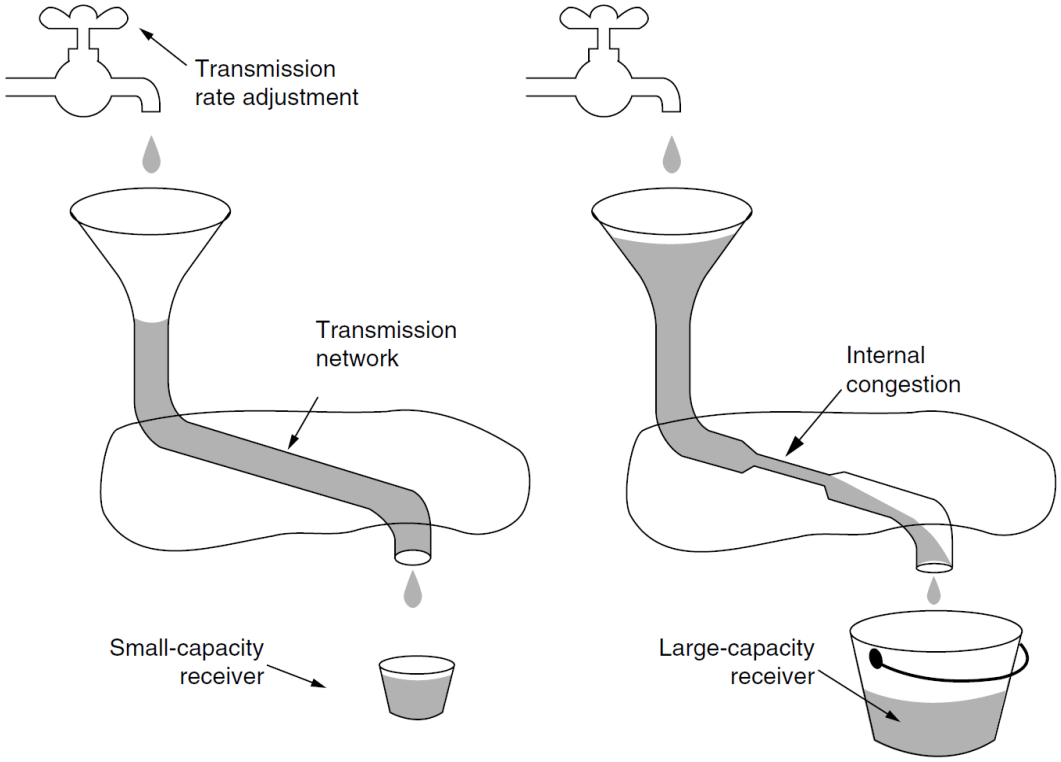


Congestion Control

- Flow Control vs. Congestion Control
  - Flow control is an issue for point-to-point traffic, primarily concerned with preventing sender transmitting data faster than **receiver** can receive
  - Congestion control is an issue affecting the ability of the **subnet** to actually carry the available traffic, in a global context

Congestion Control

- Flow Control vs. Congestion Control



Congestion Control

- Approaches to congestion control
  - **Provisioning**: add resources, upgrade links and routers
  - **Traffic-aware routing**
  - **Admission control**: do not set up a new virtual circuit if congested
  - **Load shedding**: drop packets when other congestion control mechanisms fail. In order to reduce impact, applications can mark certain packets as priority to avoid discard policy.

Congestion Control of TCP

- Two different problems exist: network capacity and receiver capacity
- Receives congestion feedback from network layer and control the traffic rate
- The sender maintains two windows, each regulates the number of bytes the sender can transmit
  - Flow control window: the size of receiver's buffer
  - Congestion window: the number of bytes the sender can have in the network at any time
  - **Maximum transmission rate** is determined by the *minimum of flow control window and congestion window*

Congestion Control of TCP

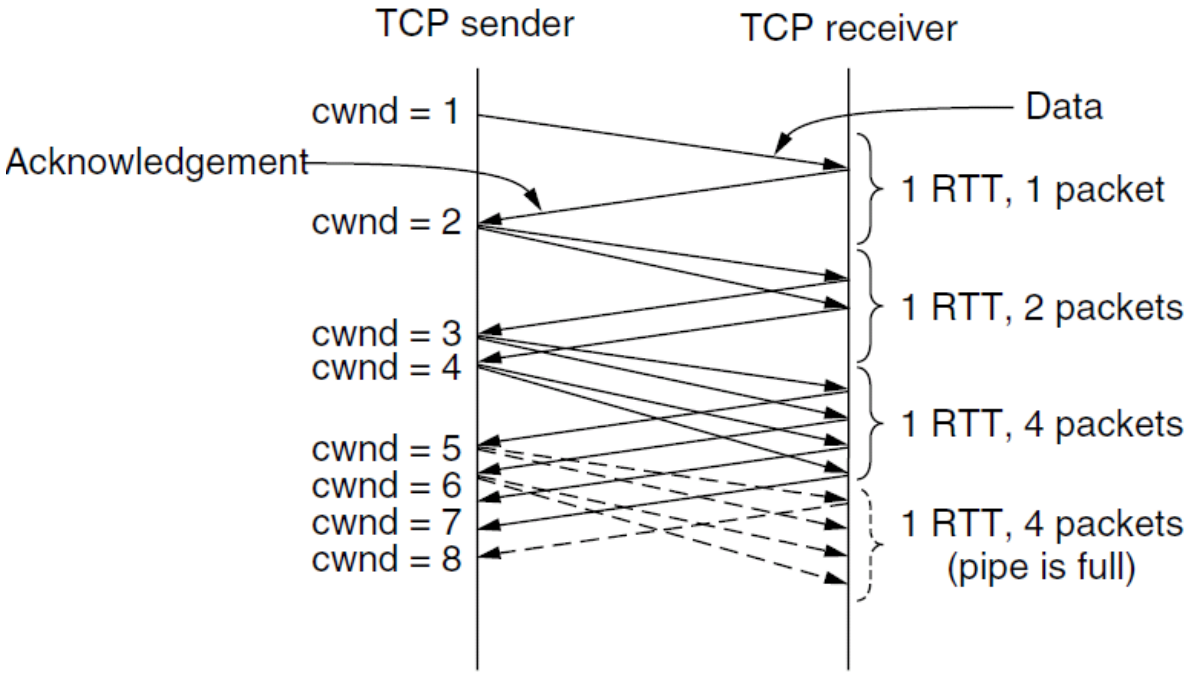
- Congestion signal: packet loss
  - detected by duplicate acknowledgements
  - assuming that the loss is caused by congestion
- Adjust congestion window according to control law, Additive Increase Multiplicative Decrease
  - TCP Tahoe: slow start with additive increase
  - TCP Reno: TCP Tahoe + fast recovery

Slow Start

- Adjust the size of congestion window
  - On connection establishment, the sender **initialises the congestion window to a size**, and transmits one segment
  - If this segment is acknowledged before the timer expires, the sender **adds another segment's worth of bytes to the congestion window**, and transmits two segments
  - As each new segment is acknowledged, **the congestion window is increased by one more segment**

Slow Start

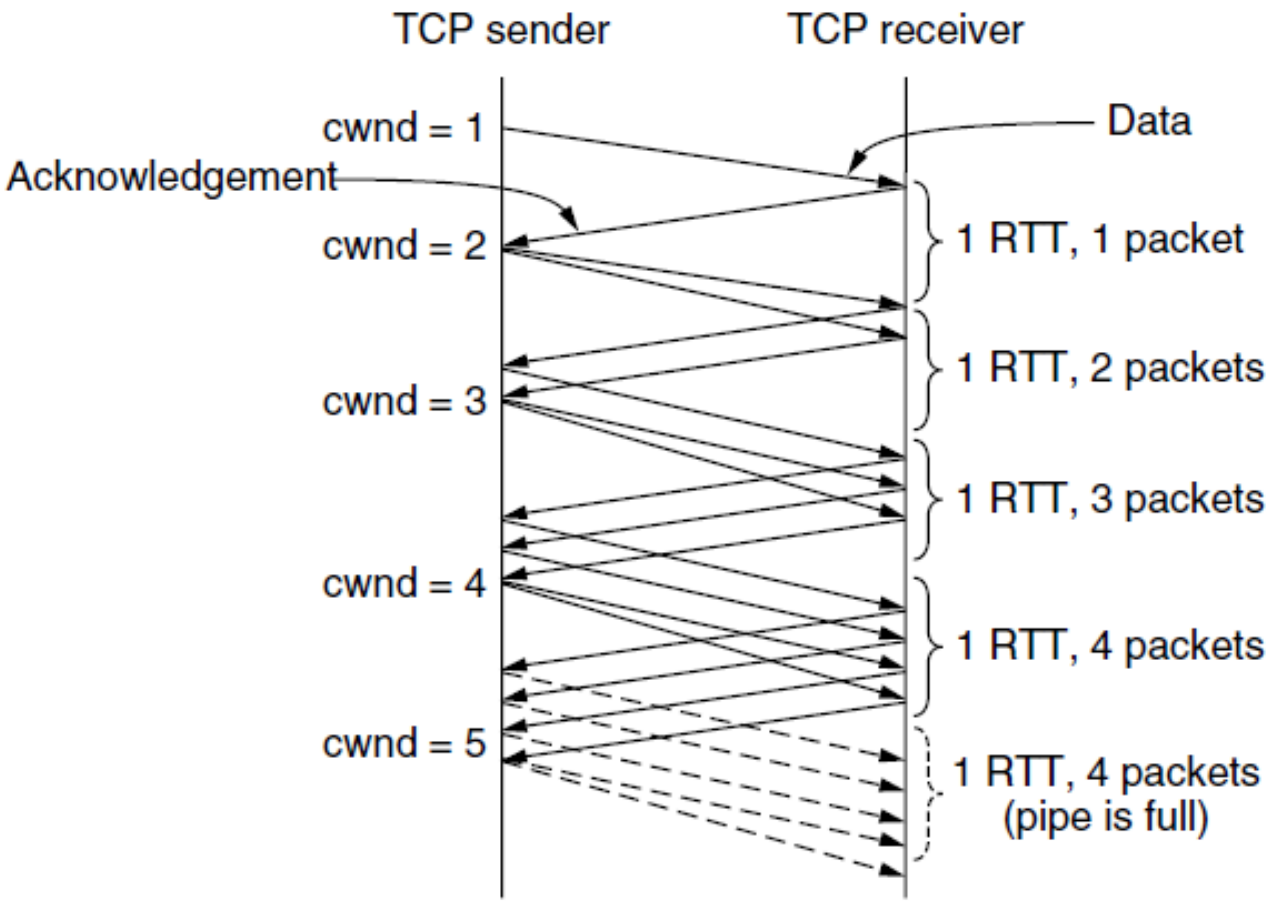
- Each set of acknowledgements **doubles the congestion window,** which grows until either a timeout occurs or the receiver's flow control window limit is reached



Slow Start

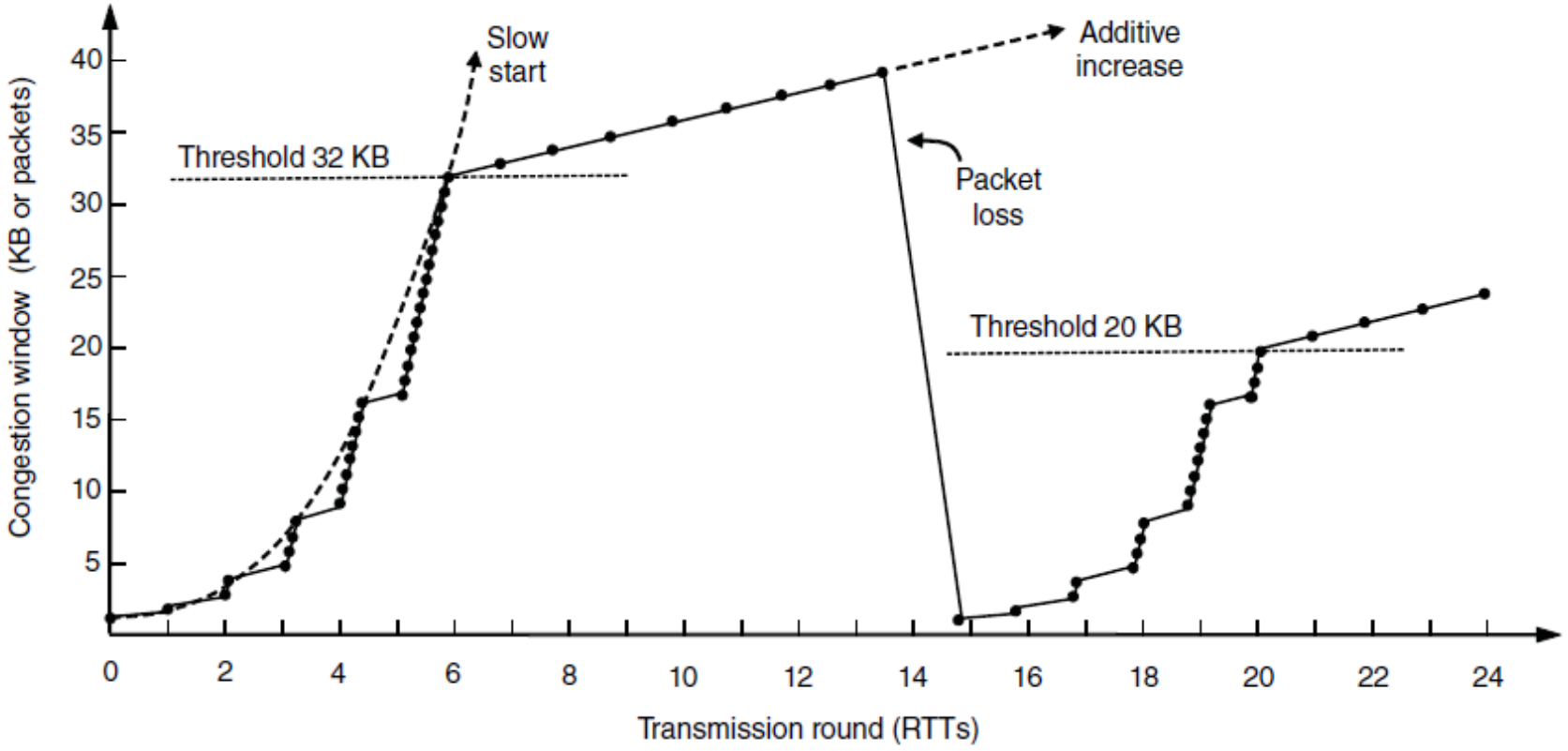
- How to control “slow start”?
  - Sender defines a **slow start threshold**, which is set to the size of receiver’s window
  - When a packet loss is detected
    - Set **slow start threshold** as **half of the current congestion window** and entire process is **restarted** (reinitialise the congestion window size)
  - When a slow start threshold is crossed
    - Switch from slow start to **additive increase**, when congestion window is increased by one segment every round

Additive Increase



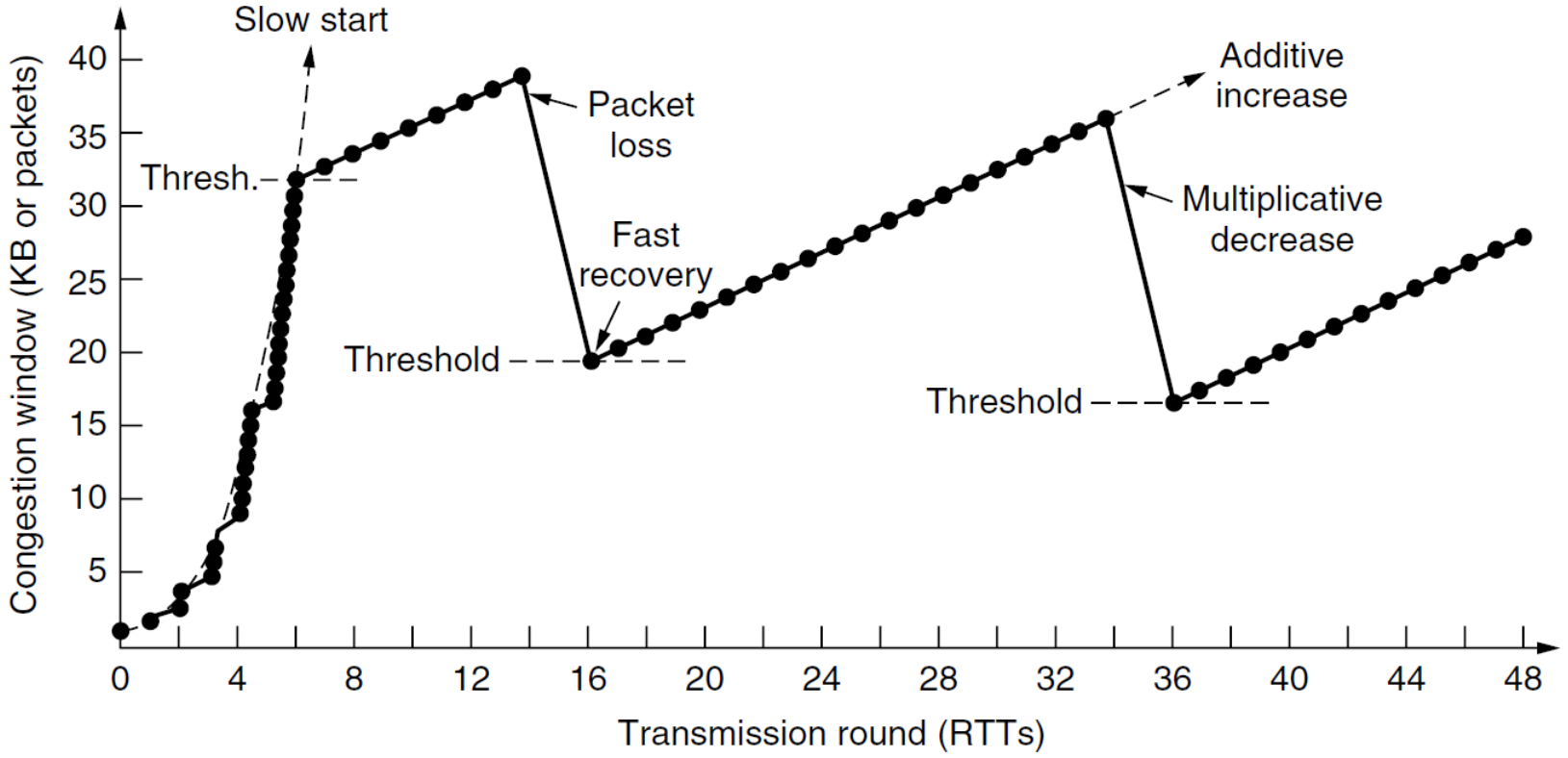
Internet Congestion Control

- TCP Tahoe: Slow start followed by additive increase; threshold is half of previous after packet loss



Internet Congestion Control

- TCP Reno: avoids slow start, except when the connection is first started or when a timeout occurs



Congestion Control of Wireless Network

- Variable capacity of wireless links
  - Signal-to-noise ratio varies when people move
  - Delay is different for Wi-Fi or Satellite
- Use packet loss as congestion signal?
  - Masking strategy: distinguish between transmission errors and congestion

Summary

- Transport Layer Service
- Elements of Transport Protocols
  - Connection establishment
  - Connection release
  - Addressing
- Programming using Sockets
- Transport Layer Protocols: UDP and TCP
- Quality of Service
  - Jitter control
  - Congestion control
  - TCP Tahoe, TCP Reno